

Injection labs

7BUIS022W

PortSwiger

Week 7-9

Lab Objectives

- **You are expected to complete this lab in three weeks.** For each section, you should start by completing the Burp Suite Academy labs before moving to the OWASP VM ones.
 1. SQL Injection
 2. NoSQL Injection
 3. OS Commands injection
 4. XML external entity (XXE) injection
 5. Server-side template injection
- Note: Some of those vulnerabilities might not apply to either OWASP or the Juice Shop but it is a good idea to understand them.
- Once this is done, you will be provided another lab document with what you need to complete for the OWASP machine.

Part A- Before Class activities: Burp Suite Academy

1 SQL Injection

- You are expected in this section to complete the following:
 - Before starting the labs: Visit [SQL Injection page](#)
- Complete the following labs:
 1. [SQL injection vulnerability in WHERE clause allowing retrieval of hidden data](#)
 2. [Lab: SQL injection vulnerability allowing login bypass](#)

3. [SQL injection attack, querying the database type and version on Oracle](#)
 4. [Lab: SQL injection attack, querying the database type and version on MySQL and Microsoft](#)
 5. [Lab: SQL injection attack, listing the database contents on non-Oracle databases](#)

 6. [Lab: SQL injection attack, listing the database contents on Oracle](#)
 7. [Lab: SQL injection UNION attack, determining the number of columns returned by the query](#)
 8. [Lab: SQL injection UNION attack, retrieving data from other tables](#)
 9. [Lab: SQL injection UNION attack, retrieving multiple values in a single column](#)
 10. [Lab: Blind SQL injection with conditional responses](#)
 11. [Lab: Blind SQL injection with conditional errors](#)
 12. [Lab: Visible error-based SQL injection](#)
 13. [Lab: Blind SQL injection with time delays](#)
 14. [Lab: Blind SQL injection with time delays and information retrieval](#)
 15. [Lab: Blind SQL injection with out-of-band interaction](#)
- **After the class-** Complete the following labs:
 1. When you are querying the database type and version, you will need to attempt your queries in different formats since you do not know what type of database is behind.

2 NoSQL Injection

- You are expected in this section to complete the following:
 - Before starting the labs: Visit [NoSQL Injection page](#)
- Complete the following labs:
 1. [Lab: Detecting NoSQL injection](#)
 2. [Lab: SQL injection vulnerability allowing login bypass](#)
 3. [Lab: Exploiting NoSQL operator injection to bypass authentication](#)
 4. [Lab: Exploiting NoSQL injection to extract data](#)

- **After the class-** Complete the following labs:
 1. When you are querying the database type and version, you will need to attempt your queries in different formats since you do not not know what type of database is behind.

3 OS Command Injection

- You are expected in this section to complete the following:
 - Before starting the labs: Visit [OS command injection](#)
- Complete the following labs:
 1. [Lab: OS command injection, simple case](#)
 2. [Lab: Blind OS command injection with time delays](#)
 3. [Lab: Blind OS command injection with output redirection](#)
 4. [Lab: Blind OS command injection with out-of-band interaction](#)
 5. [Lab: Blind OS command injection with out-of-band data exfiltration](#)
- **After the class-** Complete the following labs:
 1. When you are querying the database type and version, you will need to attempt your queries in different formats since you do not not know what type of database is behind.

4 XML external entity (XXE) injection

- You are expected in this section to complete the following:
 - Before starting the labs: Visit [XML external entity \(XXE\) injection page](#)
- Complete the following labs:
 1. [Lab: Exploiting XXE using external entities to retrieve files](#)
 2. [Lab: Exploiting XXE to perform SSRF attacks](#)
 3. [Lab: Blind XXE with out-of-band interaction](#)
 4. [Lab: Blind XXE with out-of-band interaction via XML parameter entities](#)
 5. [Lab: Exploiting XXE via image file upload](#)
- **After the class-** Complete the following labs:

1. When you are querying the database type and version, you will need to attempt your queries in different formats since you do not know what type of database is behind.

5 Server-side template injection

- You are expected in this section to complete the following:
 - Before starting the labs: Visit [Server-side template injection page](#)
- Complete the following labs:
 1. [Lab: Basic server-side template injection](#)
 2. [Lab: Basic server-side template injection \(code context\)](#)

Part B- In-Class activities: OWASP

1 SQL Injection

1. How to Detect SQL vulnerabilities (Read more [here](#))
 - The first step for a security analyst or a malicious hacker to test for SQL injections vulnerabilities is to find whether the database is actually vulnerable or not.
 - Giving a payload that can throw an error and observe what happens most often give clues.
 - For example. visit the Damn Vulnerable Web App (DVWA) on OWASP.
 - Click on SQL Injection, you will be asked to find a user ID.
 - a) Fill the userID with a single quote character `'`. Observe the error!
 - b) Give an incomplete Boolean condition or a condition that cannot be satisfied. For example `OR '1'='1`
2. Retrieving hidden data
 - Let's see if the database retrieve data that are not intended to be shown.
 - Stay on the same page from the previous step.
 - Start BurpSuite and capture the get ID for a user.
 - On BurpSuite, send the capture to the repeater
 - Now let's see if the database presents us with information that we shouldn't see
 - Let's make this statement always true

- a) Fill the userID with a single quote character **1'or'1'='1**. Observe the response on BurpSuite
- b) You can also send the response to the browser. Right click on the response and select send to the browser

3. Subverting application logic

- Let's look at a simple login page and see if we can login with any username that exists even if we don't know the password.
- We will be subverting the application logic by ensure that password value is always true.
- Browse to OWASP and select OWASP Bricks
- Now select Bricks, and then login pages
- Let us try to subvert the application logic and attempt to login as an administrator
- While BurpSuite is running and capturing requests, try to login using **administrator** and **password**
- We are not sure if the user **administrator** exists.
- In a login page, the SQL statement is usually ones that needs two inputs and both needs to be true.
 - a) Let us try first with the username **administrator** and the password logic to be always true **1'or'1'='1**
 - You will notice that you cannot login. It means that the username administrator does not exist
 - b) Let us try now with the username **admin** and the password logic to be always true **1'or'1'='1**
 - You will notice that you successfully logged in.
 - c) Another way, in this instance to subvert the logic of the application is to create and SQL statement that ignores the second part of the boolean function. We can force the SQL query to ignore anything after the username.
 - Try to put the username as **admin'--** and password **'**
 - You can see we also are able to login. the SQL compiler is only reading admin (which is true, the user exist) and ignoring anything after.

4. SQL injection UNION attacks

- Let us go back to the Damn Vulnerable Web App (DVWA) on OWASP and click on SQL Injection, you will be asked to find a user ID.
 - a) We want to query two columns and see if the database responds.

- b) Let's try to join by the union statement columns 1 and 2 together. Change the payload of the UserID to `1' union select 1,2 -- '`
 - c) You will see that the database responds by showing us Two values.
 - d) Let's see if there are three columns , for example we can try 1 and 3 `1' union select 1,3 -- ?`. You will find there are no changes. It is most likely that there are only two columns.
 - e) We can explore this further
5. We can identify the number of columns by changing the payload to add a parameter for each "Supposed to have" column.
 - a) For example and going to the same DVWA SQL injection, we can use the union select but this time add parameters in each column.
 - b) In the previous step, we were sure it is two columns, so let's start with two: `' UNION SELECT NULL,'a',NULL--`
 - c) See if the second column also contains data of type string.
 - d) You can also see if there are three columns (although at this stage, we already know there are only two). Try this `' UNION SELECT NULL,'a',NULL--`
 - e) We can explore this further
6. Using a SQL injection UNION attack to retrieve interesting data: Examining and Listing contents of the database
 - In the Subverting logic activity, we managed to login even though we did not know the password. We did this by forcing the query to ignore anything after `--`
 - Let's try a different approach. Using the Union statement.
 - Browse to OWASP and select DVWA application
 - Now select **SQL injection**
 - Let us try first to identify the name of the database. let's look specifically for columns containing the word users `1' union select table.schema, table.name FROM information_schema.tables WHERE table.name LIKE '%user%' -- '`
 - This tells us that the name of the database (**information_schema**) is **dvwa**. it also tells us that the usernames column is called **users**.
 - We now know what the passwords column is called, we can extract the usernames and passwords by joining them using a Union `1' union select user, password FROM dvwa.users -- '`
 - Now you have all usernames and passwords hashed.
7. Add Data with SQL Injection

- Log into WebGoat and go to Injection Flaws, and select "**Add Data with SQL Injection**"
 - Let's first identify if the form is vulnerable to SQL Injections. Enter the statement **jsmith' or '1'='1**
 - Now, let's try to add ourselves with a nice salary to the database. Enter the statement **jsmith'; insert into salaries values ('yourname', 10000); --**
 - Now enter the userid you used to insert into the salaries table (yourname)
 - You have successfully added an entry to the database
8. There are many other payloads you can try for both SQL injection and NoSQL injection. Have a look at this collection of SQL injection [payloads on GitHub](#) and see how they work on your assessment scenario

1.1 Blind SQL Injection

1. Blind SQL injection with WebGoat
 - Browse to <http://192.168.56.111/WebGoat> and log in with webgoat as both the username and password.
 - Click on Start WebGoat to go to WebGoat's main page.
 - Go to **Injection Flaws** and select **Blind Numeric SQL Injection**.
 - As you can see, the goal of the exercise is to find the value of a given field in a given row.
 - Let's see first how it works. Leave 101 as the account number and click **Go**. You can see that the account is valid.
 - Let's try account **1000**. We get a confirmation that the account is Invalid.
 - Up to now, we have seen the behaviour of the application, it only tells us if the account number is valid or not.
 - Let's try an injection as it is looking for numbers and probably using them as integers to search. We won't use the apostrophe in this test so enter **101 and 1=1** and press **Go!**
 - Now let's try with the account 101, which we know it exists but with a false statement such as 1=2. Enter **101 and 1=2** and press **Go!**
 - It looks like we have a blind injection here, injecting true statement results into a valid account, with a false one the Invalid account number message appears.
 - Let's see if we can discover the name of the user connecting to the database, so we first need to know the length of the username.
 - Start BurpSuite

- Let's try one, inject: **101 AND ((SELECT pin FROM pins WHERE cc_number='1111222233334444') =1000);**
 - Now we obtain an Invalid account number however we know that both the account 101 and the credit card number are correct. This leaves us with the pin number that is incorrect.
- On Burpsuite, find the POST packet with the wrong pin number attempt.
- Send it to the Intruder
- Now on account number, delete everything next to it (leaving the submit button) add this statement **101 AND ((SELECT pin FROM pins WHERE cc_number='1111222233334444') =1000);**
- Select all the statement and press **Ctrl U** to encode the statement.
- Select the value 1000 and add it as a payload by pressing **Add \$**
- Now let's modify the payload.
 - On the payload window, set the payload type to numbers
 - Set the payload settings from 2250 top 2500
 - * What we want to do is to create a payload that increments by 1 to try all pin numbers (4 digits). Now obviously this will take a very long time, so I am going to restrict the payload numbers between 2250 and 2500. I know that the pin is somewhere between these two values. In normal situations, you would not know and you will have to try more combinations to succeed.
 - Click on the settings tab in Intruder
 - I want to capture two values valid or invalid. In Grep match, clear all values.
 - * Add **Account number is valid.**
 - * Add **Invalid account number.**
 - I want Burpsuite to redirect the browser once I have answer
 - * Change **Follow Redirections to Always**
 - * Tick Process **cookies in redirection**
 - Go back to positions and and click **Start Attack**
 - Observe the attack responses. look specifically for one that return true for Valid account number. The payload is the pin number
 - Try it and submit your response to solve the activity.

2 OS Command Injection

1. OS commands injection allow you to inject commands in a form. This essentially will be two parts- the first part is where the form is executed to process the query or the input that it is designed for, then it will read the
 - Browse to `http://192.168.56.111/bWAPP/` and log in with bWAPP with username bee and password bug
 - Choose **OS command injection**
 - As you can see, the goal of the exercise is to inject commands in a normal lookup query field.
 - We can try with a simple injection to find who is the host. We will use the pipe separator `|` since it works both for windows and UNIX system, simply because we don't know which operating system is behind this application.
 - So within the DNS lookup field, we will add the command `whoami`
`www.cwscenario.site | whoami`
 - At this stage, our payload is simply a query to find which user is actually executing the command in the back-end. We can see that `whoami` returned `www-data` which is simply a username in Unix OS used by web servers.
 - At this stage as well we have been using the form element to add commands. In theory, we should be able to execute any command we want, as long we have the privilege. We might even be able to escalate the privilege from `www-data` to someone else!
2. Start BurpSuite and let's see what other payloads we can introduce to exploit the OS command injection
 - Try some of them on the OS commands injections pages either in bWAPP or in DVWA (Command execution)
 - For example
`192.168.56.111 | cat /etc/passwd`
`192.168.56.111 | netstat -an`
 - Other than being able to execute commands, what else were you able to identify from netstat?
3. OS Blind Command injection
 - There are many other payloads you can try for injecting commands. Have a look at this collection of OS command Injection [payloads on GitHub](#) and see how they work on your assessment scenario

3 XML external entity (XXE) injection

1. XML external entity injection allow hackers to process XML entities externally. This essentially mean that we will be able to parse XML queries into the system

- For this activity, we will start with Mutillade II. Browse to the application and you will find OAWSP vulnerabilities from 2007 to 2013. In fact, we have seen that those vulnerabilities are still in the top 10 in 2021. it is now labelled in as a security misconfiguration is in rank number 5 (A05) in the top 10 in 2021 see this [link](#).

2. On Mutillade, select OWASP 2013, choose A1 Injection (other) and Select **XML validator** in the XML External Entity section

- Let's try with a genuine XML entry, using the example provided
- For example

```
<somexml><message>Hello World</message></somexml>
```

- and the text content parsed from the XML submitted is a simple **Hello, World!**

- Now let's try with something more sophisticated and see if we are able to abuse the parser. In a secured application, my XML contents will be sanitised to ensure I am not injecting any malicious codes.

```
<?xml version="1.0"?>
<!DOCTYPE reading_externally1 [
<!ELEMENT variable ANY>
<!ENTITY file SYSTEM "/etc/passwd">
]>
<variable>&file;</variable>
```

- What we just did:

- * We have created an xml parser to read from a shell.
- * We declared the extenal entity named **file** that is replaced with the content of the file located at the specified system path (/etc/passwd)
- * When the XML processor encounters &file;, it replaces it with the content defined by the entity declaration (/etc/passwd).

- We can see that the contents of the file passwd are parsed from an external entity into our XML validation form.

- Now let's attempt to open a shell instead! I want to use a PHP reverse shell, I already know that PHP is running on OWASP

- <?xml version="1.0"?>
<!DOCTYPE write_On_Shell [
<!ELEMENT mycommand ANY>

```
<!ENTITY file SYSTEM "php://filter/read=convert.base64-encode/resource=
data:text/plain;base64,echo%20Hello%20from%20the%20command">
]>
<mycommand>&file;</mycommand>
```

– What we just did:

- * We have created an xml parser to write to the a shell.
- * `php://filter/read=convert.base64-encode/resource=data:text/plain;base64,echo%20Hello%20from%20the%20command`: This PHP code is URL-encoded. It executes the command `echo Hello from the command` and then encodes its output in base64 format.
- * `php://filter` is a stream wrapper that allows you to filter data through a chain of filters.
- * `convert.base64-encode` is a filter that converts data to base64 encoding.
- * When this XML is parsed, the PHP code will be executed, and its output will be included as the value of the entity `file`, which will then be substituted into the `mycommand` element.

4 Other Injections

- On your own time, explore other injections on Mutillidae. You will find HTML injection quite interesting. We will come back to the HTML injection in the next lab when we conduct Cross Site Scripting attacks.